

IMPERIAL COLLEGE LONDON

TIMED REMOTE ASSESSMENTS 2021-2022

BEng Honours Degree in Computing Part II
MEng Honours Degrees in Computing Part II
BEng Honours Degree in Mathematics and Computer Science Part II
MEng Honours Degree in Mathematics and Computer Science Part II
for Internal Students of the Imperial College of Science, Technology and Medicine

*This paper is also taken for the relevant assessments for the
Associateship of the City and Guilds of London Institute*

PAPER COMP50004

OPERATING SYSTEMS

Monday 16 May 2022, 10:00

Writing time: 90 minutes

Upload time: 25 minutes

Answer ALL TWO questions

Open book assessment

This time-limited remote assessment has been designed to be open book. You may use resources which have been identified by the examiner to complete the assessment and are included in the instructions for the examination. You must not use any additional resources when completing this assessment.

The use of the work of another student, past or present, constitutes plagiarism. Giving your work to another student to use constitutes an offence. Collusion is a form of plagiarism and will be treated in a similar manner. This is an individual assessment and thus should be completed solely by you. The College will investigate all instances where an examination or assessment offence is reported or suspected, using plagiarism software, vivas and other tools, and apply appropriate penalties to students. In all examinations we will analyse exam performance against previous performance and against data from previous years and use an evidence-based approach to maintain a fair and robust examination. As with all exams, the best strategy is to read the question carefully and answer as fully as possible, taking account of the time and number of marks available.

Paper contains 2 questions

- 1 a An operating system (OS) supports *kernel-level* threads with preemptive scheduling. User programs can use a mutual exclusion lock (*mutex*) *m* of type `struct pthread_mutex_t` with the following three functions:

```
pthread_mutex_init(&m)
pthread_mutex_lock(&m)
pthread_mutex_unlock(&m)
```

- i) Briefly explain the purpose of *synchronisation primitives* in operating systems and give three examples of synchronisation primitives.
 - ii) Explain what information the OS kernel must associate with the type `struct pthread_mutex_t`.
- b The above OS executes a user program with two threads, *T1* and *T2*, which both execute the code below. *T1* is currently executing line 7 and *T2* has just called the function in line 6. The function `perform_expensive_computation()` takes 2 secs to complete.

```
1
2 pthread_mutex_t m;
3 int buffer[BUFFER_SIZE];
4 unsigned int next_pos = 0;
5
6 void critical(int a) {
7     pthread_mutex_lock(&m);
8     perform_expensive_computation();
9     buffer[next_pos] = a;
10    next_pos = (next_pos + 1) % BUFFER_SIZE;
11    pthread_mutex_unlock(&m);
12 }
```

- i) Describe, line-by-line, a likely execution trace for the two threads, explaining any system calls and actions taken by the OS scheduler.
- ii) Propose and justify a change to the above program that improves performance without compromising correctness.
- iii) Explain why the above synchronisation approach would not work if the OS only supported *user-level* (green) threading.
- iv) Propose a solution to the problem that you have identified under iii).

The two parts carry, respectively, 25% and 75% of the marks.

- 2 We have a data processing system with the following characteristics:
- Cache access latency: 0.5 nsec
 - Main memory access latency: 15 nsec
- a We have an application with a cache hit ratio of 90%. Calculate the effective access time (EAT) for memory that we should expect to see.
- b After running the same application, we measure a different EAT of 2.04 nsec, because we did not take virtual memory into account above. We then get a CPU with a new MMU hardware implementation that defines a modified page table format, promising us an EAT of 1.99 nsec; the rest of the CPU is the same as we have been using until now. Do we need to change any of our software to take advantage of this new CPU MMU improvements? Briefly explain why.
- c After a close look, we realize that all the code and data the application needs fits in main memory, and also that we will never run other applications on our system. What is the simplest possible virtual memory translation mechanism that we can use that will give us the lowest possible EAT? Briefly describe why.

We now look at an application that needs 100 GiB of data that is stored on an input file, where we only have 1 GiB of main memory.

- d We installed a SSD device to store all our input data, using an *ext2* file system configured with 8192 Byte blocks and 8-Byte indirection pointers. Calculate the maximum number of disc block access operations we will need in the worst case to read an arbitrary position in the file. Show your working, assuming our first access is always to the inode structure.
- e We have moved our file system from the SSD into a newer NVM (non-volatile memory) device. The NVM can be accessed as a regular block storage device (like the SSD), but also using memory-mapped I/O. Applications can thus use `mmap` to map the input file contents directly into the NVM without exhausting the main memory. Since both on-demand paging and ahead-of-time page table mapping take time, briefly describe the best OS strategy that decides when and how to map these contents, assuming we know the application's access pattern ahead of time:
- i) The application accesses the entire file sequentially.
 - ii) The application accesses the entire file randomly.
 - iii) The application accesses an unknown subset of the file randomly.

The five parts carry, respectively, 15%, 15%, 20%, 30%, and 20% of the marks.